Live stacking in the Starfront N.I.N.A. plugin

Merging the Rapid Stacker Collab upload engine into the Starfront N.I.N.A. plugin, behind an on/off setting, without disturbing Starfront's existing logic.

Context

Starfront's N.I.N.A. plugin (`starfront/nina/`) deals sky to telescopes, shoots it through the *Starfront collaboration* sequence container, credits every saved light to its panel in a JSON ledger and reports the totals to collab.starfront.space. It never touches the frame file: the only file-collection feature is a `collectFiles` project flag that opens a web page where people upload by hand.

Rapid Stacker Collab's N.I.N.A. plugin (`Rust/rapid-stacker-collab/nina-plugin/`) does the opposite half: it signs a contributor in by device code, joins a project by join code, and uploads each saved frame to S3 through presigned multipart URLs from the Collab API, with a restart-safe SQLite queue, an upload policy, a bandwidth cap and location stripping. A worker then folds each frame into the project's live per-filter stacks.

The author is providing the stacking component of Starfront's collaboration pipeline. The outcome: a Starfront user turns on *Live stacking* in the plugin options, the coordinator puts a Collab join code on the Starfront collaboration, and every light that telescope shoots for that collaboration is uploaded and stacked as it lands. With the setting off the plugin behaves exactly as today.

Decisions already taken with the user (settled):

Question	Decision
How a frame finds its Collab project	The Starfront project record carries the Collab **join code** (coordinator enters it in the start/edit form, the `collectFiles` precedent). The plugin learns it at check-in and auto-joins.
Credentials	A separate Collab device-code sign-in on the Starfront options page; the `rscu_` token is DPAPI-wrapped in Starfront's `settings.json` beside the Starfront tokens.
Code reuse	Copy `RapidStackerCollab.Core` (NINA-free, MPL-2.0) into `starfront/nina/src/` as a second assembly shipped in the zip, with its tests. The Shell is not copied; the few pieces needed are small ports.
UI scope v1	A *Live stacking* section on the options page plus a short status block in the existing Starfront dock panel. No sequencer items, no backfill, no second dockable.
Backend	The Collab API (`https://collab.astro.ekeservices.com/api/` , torn down 2026-10-16, redeployable), base URL is a setting.

Both repos are clean. Starfront is on `main` ; rapid-stacker-collab is on `fix/plugin-test-teardown-retry` (one test-only commit ahead of `master`). Nothing is pushed, released or deployed by this work: Starfront plugin releases and server deploys are the author's call (`starfront/CLAUDE.md`).

Findings that shape the design

- **Hook point exists.** `FrameReporter.Lookup(targetName)` (`nina/src/Starfront.Collab/Services/FrameReporter.cs:70`) is public and returns the `PanelContext` (Starfront `ProjectId`) for any saved

light the container shot. A second `ImageSaved` subscriber can resolve the project the same way `OnImageSaved` does, so `FrameReporter` stays untouched and the ledger path is unchanged.

- **Check-in already delivers project data.** `StarfrontSession.PollAsync` refreshes `Projects` from `GET /api/v1/agent/projects` every 10 minutes and raises `Updated`; the dock already subscribes to it. The join code rides on that listing; no change to `StarfrontSession.cs`.
- **Starfront ships one DLL and nothing else.** `build/Plugin.targets` is an allow-list; `Stage-Plugin.ps1` creates sub-directories per destination, so `runtimes/win-x64/native/e_sqlite3.dll` packages without script changes. No SQLite anywhere today.
- **`build/Build-Release.ps1` L27-28 globs `src/**/*.csproj` and throws unless exactly one exists.** A second project breaks the release script unless it is pointed at `src/Starfront.Collab/Starfront.Collab.csproj`.
- **`nina/Directory.Build.props` forces `net8.0-windows` + `UseWPF` on the whole tree** (Nullable off, warnings not errors). Core is `net8.0`, Nullable on, warnings-as-errors, MPL header enforced via `.editorconfig`. A nested `Directory.Build.props` and `.editorconfig` under the Core folder keep each side's rules; the copied csproj stays byte-identical to upstream.
- **Core's object graph is in the Shell** (`CollabServices.cs`, 346 lines): it is the pattern for the new `StackingUploader`, not code to copy. `FrameIntake.Evaluate` does its own OBJECT/footprint attribution; Starfront already knows the project, so only the sniff/read/classify/SKIP part is ported.
- **N.I.N.A. itself uses `System.Data.SQLite`**, a different assembly from `Microsoft.Data.Sqlite`, so the two do not collide (verified by the Collab plugin's restore). Loading the native `e_sqlite3.dll` from inside a plugin is still unverified on either side.
- **The Collab membership's filter mapping is applied server-side at `frames/init`.** The plugin needs a mapping only to join and to skip lights mapped to `SKIP`. The mapping can be built headlessly from the wheel's filter names with Core's `FilterNormalizer`, using Starfront's per-profile `FilterMap` as the override.

Design decisions

Topic	Decision and reason
Naming	*Live stacking* on screen; namespace `Starfront.Collab.Services.Stacking`; log prefix `Starfront stacking:`.
Server field	`stackingJoinCode` on the project payload, normalised to `XXXX-XXXX-XXXX`. The listing carries `stackingLinked` (bool) for everyone and the code itself only to telescopes that are on the project (have a task) or whose owner owns it. The code admits anyone to the Collab project, so a browsing telescope must not see it; the owner's telescopes must, or the Edit form (which loads from the agent listing) would blank it on save.
Which frames	Only lights the Starfront container shot (`Lookup` hit) for a project with an active link. Non-Starfront lights are never uploaded in v1.
Calibration frames	When *Also upload darks, flats and bias* is on, routed to the Collab project of the most recent light this session uploaded; skipped with one log line when there is none. Calibration carries no target and is shot outside the container; the last light is the only honest proxy. Documented.
`membership_pending`	Frames queue; Core's runner pauses the project (`ProjectBlocked`, rechecks every 15 min) and the maintenance loop re-reads the membership. Nothing shot tonight is lost.
`terms_outdated`	Link status `terms_outdated` with the terms URL; the options page shows *Accept and rejoin*; no upload until clicked. The first join accepts the resolved `terms_version` headlessly and the enable toggle's hint says so.
Upload modes	Immediately / When idle / Time window. `AfterSequenceEnd` is dropped: it needs `ISequenceMediator` (a new MEF import in three places plus deferred subscriptions that throw until the sequencer registers). Imaging state comes from `ICameraMediator.RegisterConsumer`, already in `HostServices`. The container's `finally` does not force an upload: that would override a time window chosen for a slow link. *Upload now* stays a button.
SQLite	Keep upstream's `Microsoft.Data.Sqlite` + `bundle_e_sqlite3` and the `NativeLibrary.Load` preload by full path (N.I.N.A. adds a plugin's native search path only after `Initialize`). `bundle_winsqlite3` would avoid shipping a native DLL but does not run on macOS, so the Core tests would diverge from upstream. It is the fallback if the checklist shows the preload failing inside N.I.N.A.
Preload timing	Inside `StackingUploader.Start()` right before the queue is opened, so nothing native is touched while the setting is off.
Settings home	Global `settings.json` for everything about the person and PC (token via the existing `Protected`); per-profile only `StackingRigId`, because the rig is the equipment. Mapping and membership live in the link file.
Link store	`DataFolder/stacking/links.json`, Newtonsoft, `.tmp` + move, `.broken` on corrupt: the `settings.json` / `ledger.json` conventions.
Lifetime	`Runtime.Stacking` is constructed in `Ensure` (no I/O) and started only when `StackingEnabled`. Toggling the setting starts or stops it at runtime through `Settings.PropertyChanged`. Off means no `ImageSaved` subscription, no camera consumer, no `Updated` subscription, no folder, no SQLite, no HTTP.

Phase 0: the review PDF

Before any code, the plan was rendered to PDF for the author's review;
`STARFRONT_LIVE_STACKING_PLAN.pdf` beside this file is that rendering.

Phase 1: Starfront server carries the join code

Modify `starfront/server/app.py`

- Add `_stacking_join_code(text) -> str` near `ProjectRequest` (~L417): strip, upper-case, drop spaces and hyphens; `""` passes through; otherwise require 12 characters from the Crockford alphabet `23456789ABCDEFGHJKMNPQRSTUVWXYZ` (upstream `JoinFlow.CodeAlphabet`) and return `XXXX-XXXX-XXXX`; else `HTTPException(422, "a Rapid Stacker Collab join code is twelve characters, like H4RT-92BK-WNPC")`. Comment: it is a credential, not a label.
- `ProjectRequest` (L431 area): `stackingJoinCode: str = ""`; `create_project` payload (L1646): `"stackingJoinCode": _stacking_join_code(body.stackingJoinCode)`.
- `ProjectUpdateRequest` (L1665): `stackingJoinCode: str | None = None`; `update_project` (L1697): set when not None; empty string clears.
- `open_projects` listing (L735-785, beside `collectFiles` at L771): `code = payload.get("stackingJoinCode") or ""`; emit `"stackingLinked": bool(code)` always and `"stackingJoinCode": code` only when the agent has a task on the project or its owner is the project's `owner_id` (check the agent record's owner key in `store.py` before relying on it), else `""`.

Modify `starfront/tools/check_server.py` (beside the `collectFiles` cases, L2017-2032): create with `" h4rt 92bk-wnpc "` stores `H4RT-92BK-WNPC`; a code containing `1` is 422; the owner's agent sees the code without joining, a joined agent of someone else sees it, a third agent sees `""` with `stackingLinked: true`; `PUT` with `""` clears; `PUT` without the field leaves it.

Verify: `cd starfront && python tools/check_server.py` passes, new cases included.

Phase 2: copy Core, build, package

Create `nina/src/RapidStackerCollab.Core/`: copy upstream `nina-plugin/src/RapidStackerCollab.Core/` from rapid-stacker-collab `master` (all `.cs`, `Filters/aliases.json`, `Filters/ALIASES.SOURCE.md`, the csproj; not `bin/`, `obj/`, `.DS_Store`). No local edits to the copied files.

- New `Directory.Build.props` in that folder: import the parent (`$([MSBuild]::GetPathOfFileAbove('Directory.Build.props', '$(MSBuildThisFileDirectory)../'))`), then `UseWPF=false`, `Nullable=enable`, `TreatWarningsAsErrors=true`, `WarningLevel=9999`, `EnforceCodeStyleInBuild=true`, upstream's Authors/Company/Product/Copyright (EKE Services LLC), `Version` default `0.1.0.0`. A `-p:Version=1.0.0.N` global property still flows in.
- New `.editorconfig` (copy of upstream's, `root = true`): MPL header rule scoped to this folder; Starfront's `.cs` files stay header-free.
- New `SOURCE.md`: upstream path, the commit copied, the copy command, "no local edits" policy, and the list of Shell files ported by hand into `Starfront.Collab/Services/Stacking/` so a re-sync knows what else to diff.

Create `nina/tests/RapidStackerCollab.Core.Tests/`: copy upstream's test project (Fakes incl. `FakeCollabServer.cs`, Support, Contract, Assets, csproj) with the same nested `Directory.Build.props` and `.editorconfig`. `RepoPaths.Find` walks up and `Assert.Ignore`s when `docs/api/openapi.yaml`, `scripts/contract-examples.json`, `docs/api/examples/strip-vectors.json` and `worker/crates/`

`filters/*` are absent, so expect about 15 ignored tests. Optional later: vendor those files under `nina/docs/api/` and `nina/scripts/`, which `RepoPaths` would find with no code change.

Create **`nina/Starfront.sln`** with the three projects.

Modify **`nina/src/Starfront.Collab/Starfront.Collab.csproj`**: `<ProjectReference Include="../RapidStackerCollab.Core/RapidStackerCollab.Core.csproj" />`. `CopyLocalLockFileAssemblies=true` (already set) puts the SQLite DLLs and `runtimes/win-x64/native/e_sqlite3.dll` in `$(TargetDir)`.

Modify **`nina/build/Plugin.targets`** allow-list (after L24): `RapidStackerCollab.Core.dll` and `.pdb`, `Microsoft.Data.Sqlite.dll`, `SQLitePCLRaw.core.dll`, `SQLitePCLRaw.batteries_v2.dll`, `SQLitePCLRaw.provider.e_sqlite3.dll`, `runtimes/win-x64/native/e_sqlite3.dll` (same destination), and `$(PluginRoot)/NOTICE.txt`.

Create **`nina/NOTICE.txt`**: third-party notice for this zip (RapidStackerCollab.Core MPL-2.0 EKE Services LLC + upstream commit; Microsoft.Data.Sqlite MIT; SQLitePCLRaw Apache-2.0; SQLite public domain). Upstream's `NOTICE` says "the rest is proprietary", which is wrong here, so it is rewritten rather than copied.

Modify **`nina/build/Build-Release.ps1`** L27-29: replace the glob with the explicit `src/Starfront.Collab/Starfront.Collab.csproj` and an exists check.

Modify **`nina/build/Stage-Plugin.ps1`**: one line that throws if any `NINA.*` file is staged (Starfront has no reference audit; this is the cheap half of upstream's).

Modify **`nina/.github/workflows/ci.yml`**: `dotnet test tests/RapidStackerCollab.Core.Tests -c Release` after the build (the Kestrel fake runs on `windows-latest`).

Verify (macOS): `cd nina && dotnet build Starfront.sln -c Release` (WPF compiles via `EnableWindowsTargeting`, as upstream proves) and `dotnet test tests/RapidStackerCollab.Core.Tests` pass with ~15 ignored. **Verify (Windows):** `-t:PackagePlugin` zip holds exactly the allow-list and no `NINA.*`; `-t:DeployPlugin`, start N.I.N.A., the plugin loads with the setting off.

Phase 3: settings and Collab sign-in

Modify **`nina/src/Starfront.Collab/Settings/StarfrontSettings.cs`** (house style: `GlobalSettings` field + property whose setter assigns, `Save()`, `RaisePropertyChanged()`):

- Global: `StackingEnabled=false`, `StackingApiBase=""` (blank reads as `https://collab.astro.ekeservices.com/api/`, always a trailing `/`), `StackingToken` (via `Protected.Wrap/Unwrap`), `StackingTokenExpiresUtc`, `StackingUserName`, `StackingUserEmail`, `StackingDisplayName`, `StackingUploadMode="WhenIdle"`, `StackingWindowStart="06:00"`, `StackingWindowEnd="14:00"`, `StackingIdleGraceSeconds=30` (0-600), `StackingBandwidthKBps=0` (>= 0), `StackingStripLocation=true`, `StackingIncludeCalibration=false`, `StackingTempRetentionHours=24` (1-720), `StackingHistoryDays=30` (1-3650), `StackingPaused=false`.
- Derived: `StackingSignedIn`; `ForgetStackingCredentials()`; `StackingPolicy()` returning Core's `UploadPolicySettings` (unknown mode reads as WhenIdle).
- Per profile: `StackingRigId` through `options.GetValueString/SetValueString`.
- Class summary gains a sentence: a third credential, same rules as the other two.

Create `Services/Stacking/SettingsTokenStore.cs`: Core `ITokenStore` over `Settings.StackingToken`; `Clear` also blanks name, email and expiry.

Create `Services/Stacking/StackingSignIn.cs` (`BaseINPC`): `Busy`, `UserCode`, `VerificationUrl`, `Status`; `SignInAsync` = Core `DeviceSignIn.StartAsync("Starfront for N.I.N.A.")`, open the browser (https only, `UseShellExecute`), `WaitForApprovalAsync`, store name/email/expiry, `Notification.ShowSuccess`; denied/expired/failed set `Status`; `CancelAsync`; `SignOutAsync` revokes the token (ignoring `token_invalid` and network errors) then `ForgetStackingCredentials()`. Uses the uploader's `ApiClient`, so sign-in is enabled only while the setting is on.

Modify `Plugin.cs`: expose `Stacking` (a `Ui/StackingOptionsVM`) with `SignIn/SignOut/CancelSignIn/AcceptTerms/RetryLink` as `AsyncRelayCommand`s gated on `!Busy`, `NotifyCanExecuteChanged` via `Dispatch.OnUi`, as the existing commands do.

Verify: build; with the setting off, `settings.json` gains the new keys with defaults and nothing else changes. The device flow itself is covered by upstream `DeviceSignInTests` against the fake server; the N.I.N.A. side goes on the checklist.

Phase 4: links, auto-join, intake, lifetime

Create `Services/Stacking/JoinCodes.cs` (pure): `TryNormalize`, `Format`, `CleanCode`, ported from upstream `JoinFlow.cs` L384-407. Used by the form and the joiner.

Create `Services/Stacking/StackingLinks.cs`: `StackingLink` POCO (`StarfrontProjectId`, `StarfrontName`, `JoinCode`, `CollabProjectId`, `CollabName`, `Status` in active / pending / terms_outdated / needs_rig / no_filters / failed, `Message`, `TermsVersion`, `TermsUrl`, `RigId`, `DisplayName`, `RequireStrip`, `DeclaredFilters`, `Mapping`, `UpdatedUtc`, `NextAttemptUtc`) and the store (`Load/Save/Lookup/Upsert/Remove/All`, `lock`, atomic write, `.broken`). File `DataFolder/stacking/links.json`.

Create `Services/Stacking/StackingMapping.cs` (pure): `Build(wheelNames, declared, colour, starfrontFilterMap, normalizer)`, rules ported from upstream `FilterMappingGrid.cs` L107-145: a Starfront `FilterMap` delivery letter L/R/G/B/H/O/S maps to the declared canonical (`L/R/G/B/HA/OIII/SII`) when declared; otherwise `FilterNormalizer.Normalize(name, new FilterContext(declared, bayer))`; recognised-and-declared gives the canonical; anything else `SKIP`. Returns entries plus a one-line summary for the log; `AllSkipped` makes the joiner mark `no_filters`.

Create `Services/Stacking/StackingJoiner.cs`: `ReconcileAsync(IReadOnlyList<ProjectListing>)`, serialised by a `SemaphoreSlim`, only when enabled and signed in. For each listing with a non-empty `StackingJoinCode`:

1. Same code and status active: skip. Different code: re-join. `NextAttemptUtc` in the future: skip.
2. `ResolveJoinCodeAsync(code)`; `AlreadyMember` means `GetMembershipAsync` and link active with the server's mapping.
3. Rig: `Settings.StackingRigId`, else `UpsertRigAsync` + `PatchRigAsync(Confirmed)` from Starfront's own facts (`RigProfile`, profile camera/telescope settings, `SensorType` to Bayer pattern). `RigProfile.WhatIsMissing` non-null gives status `needs_rig`, retried next poll.
4. Mapping via `StackingMapping.Build` over the wheel's filter names.

5. `JoinProjectAsync(project, {Code, DisplayName = clamp(StackingDisplayName | UserName | TelescopeName, 2..40), AcceptedTermsVersion = resolved.TermsVersion, AllowOwnerRawDownload = false, FilterMapping})` ; status from `Membership.Status` ; store `RequireStrip` from the resolved policy.
6. `ApiException` : `terms_outdated` sets the status and `TermsUrl` ; `project_not_open` / `project_deleted` / `invalid_code` give failed with `NextAttemptUtc` +6 h; transient +30 min; auth failure raises `AuthRequired` . Every outcome: one log line, `Links.Upsert` , `Changed` .

- Links whose Starfront project no longer lists a code stay on disk but are not fed.
- `AcceptTermsAsync(link)` : re-resolve and join again with the new version.

Create **`Services/Stacking/ImagingState.cs`** : port of upstream `ImagingStateTracker` without the sequence members, plus `CameraStateConsumer : ICameraConsumer` (busy when `IsExposing` or `CameraState` is Exposing / Reading / Download).

Create **`Services/Stacking/StackingIntake.cs`** : `Evaluate(path, filterHint, link, includeCalibration)` ported from upstream `FrameIntake.Evaluate` L75-121 without attribution: sniff FITS/XISF, read the header, `FrameKinds.Classify` , `ShouldUpload` , a light mapped `SKIP` is skipped; `Enqueue` builds `NewQueueItem(link.CollabProjectId, path, filename, kind, header, settings.StackingRigId)` and wakes the runner when added. Never throws, never writes outside the queue.

Create **`Services/Stacking/VerdictPoller.cs`** , **`TempCleaner.cs`** , **`NativeSqlite.cs`** : ports from upstream Shell/Plugin, logging through `Logger` with the `Starfront stacking:` prefix.

Create **`Services/Stacking/StackingUploader.cs`** , the object graph and lifetime, mirroring upstream `CollabServices.cs` L45-99 and L124-192:

- Constructor takes `HostServices` , `StarfrontSettings` , `FrameReporter` , `StarfrontSession` ; loads `Links` only; subscribes `Settings.PropertyChanged` (`StackingEnabled` toggles `Apply()` ; other `Stacking*` keys re-apply the bandwidth limit and wake the runner).
- `Start()` (locked, idempotent): `NativeSqlite.Preload` from the plugin folder; create `DataFolder/stacking` ; `UploadQueue(queue.db)` + `RecoverInterrupted` ; one `SocketsHttpHandler` for `apiHttp` (60 s) and `s3Http` (infinite); `ApiClient` with `SettingsTokenStore` , `BaseUri = StackingApiBase` , user agent `Starfront-NINA/<ver>` ; `BandwidthLimiter` ; `UploadEngine` ; `UploadRunner` with `() => settings.StackingPolicy()` , `CurrentImagingState` , `TempDirectory = .../tmp` , `StripPredicate = item => settings.StackingStripLocation || Links.RequireStrip(item.ProjectRef)` ; progress / item-finished / auth-required callbacks; a `Channel<StackingFrame>` with one drain task (the `SavedFrameDispatcher` shape; `StackingFrame` = Core `SavedFrame` + link); subscribe `host.ImageSave.ImageSaved` , `host.Camera.RegisterConsumer` , `session.Updated` ; tasks: runner loop (restart after 30 s on a crash), maintenance loop (30 min: `TempCleaner.Sweep` , `Queue.Purge` , membership re-check for pending/active links), verdict loop (30 s while anything awaits a verdict). First `ReconcileAsync(session.Projects)` at once.
- `OnImageSaved` : catch-all; `Imaging.ImageSaved()` ; LIGHT: `reporter.Lookup(Target.Name)` , null means not ours; `Links.Lookup(context.ProjectId)` , null or failed is logged once per project per session; else post. Calibration: only when `StackingIncludeCalibration` , link from `reporter.LastProject` . Posts `new SavedFrame(args.PathToImage.LocalPath, args.Filter, args.Duration, args.IsBayered, args.FileType.ToString())` with the link.

- `OnSessionUpdated`: `Joiner.ReconcileAsync(session.Projects)` in try/catch; `Runner.Wake()`.
- `Stop()` (locked): unsubscribe everything, `RemoveConsumer`, cancel, wait each task up to 5 s, dispose queue and HTTP, null the graph. Bounded blocking is fine in `Teardown`.
- Public surface for the UI: `Running`, `AuthRequired`, queue counts, `Runner.LastDecision`, current item and progress, last result, `Links`, `Joiner`, `SignIn`, `PauseResume()`, `UploadNow()`, `Changed`.

Modify **`Services/Runtime.cs`**: `public static StackingUploader Stacking`; built in `Ensure` after `Session`; `Start()` calls `Stacking.Apply()` (starts only if enabled); `Shutdown()` calls `Stacking.Stop()` first, in its own try/catch, so no frame lands in a disposed queue.

Modify **`Server/Models.cs`** `ProjectListing` (L120): `[JsonProperty("stackingJoinCode")] string StackingJoinCode = ""` and `[JsonProperty("stackingLinked")] bool StackingLinked`.

Verify: build on macOS. Windows, setting off: no `stacking/` folder appears, `tasklist /m e_sqlite3.dll` shows nothing, the N.I.N.A. log has no `Starfront stacking:` line, ledger and report lines unchanged. Setting on: the Phase 6 checklist. `JoinCodes`, `StackingMapping` and `StackingIntake.Evaluate` have no N.I.N.A. types, so a future `Starfront.Collab.Tests` project can cover them; out of scope now.

Phase 5: UI

Modify **`Ui/StartCollabVM.cs`**: `StackingJoinCode` property; `Reset` blanks it, `BeginEdit` loads `listing.StackingJoinCode`, `Body()` sends the normalised code (blank allowed; invalid raises the message *A Rapid Stacker Collab join code is twelve characters, like H4RT-92BK-WNPC* through the existing `Message` path).

Modify **`Ui/DockTemplates.xaml`**: under the *Collect the lights on the server* row (L198-201) a label + `TextBox` *Rapid Stacker Collab join code* with a tooltip. On the project card (near `FilesText`, ~L375) a `TextBlock` bound to `StackingText`. After *Tonight's deal*, a *Live stacking* block (heading in the house style, four status lines, Pause/Resume and Upload now buttons), visible only when `Stacking.Visible`.

Modify **`Ui/StarfrontDockVM.cs`**: `ProjectRowVM.StackingText` (*Frames are live-stacked on Rapid Stacker Collab* plus this telescope's link status); `StackingStatusVM Stacking` created in the constructor, subscribed in `Initialize`, released in `Dispose`.

Create **`Ui/StackingStatusVM.cs`** (`BaseINPC`): `Visible`, `Headline` (counts by state), `CurrentText` (file and percent), `ReasonText` (the policy's reason for waiting), `LastText` (last result or verdict), `Hint` (sign in needed, token expiring within 14 days, terms outdated), `PauseLabel`, `TogglePauseCommand`, `UploadNowCommand`. A 2 s `System.Threading.Timer` runs only while `Runtime.Stacking.Running` and the panel is visible; updates through `Dispatch.OnUi`; also refreshes on `Stacking.Changed`.

Create **`Ui/StackingOptionsVM.cs`** and **modify** **`Ui/OptionsTemplates.xaml`**: a *Live stacking* section after *This telescope*, same heading style and three-column grid. Rows: enable (CheckBox + separate label + the terms hint); sign-in block (signed out: button, status, user code, Open browser / Cancel; signed in: *Signed in as X, token expires in N days*, Sign out); upload policy ComboBox over a `(Code, Label)` list; window start/end (visible for Time window); idle grace; bandwidth KB/s; *Remove my location from the headers*; *Also upload darks, flats and bias*; Collab server (blank for the built-in one); a *Linked collaborations* `ItemsControl` (name, Collab project, status, message, Accept new terms / Retry buttons). The existing `Starfront_ShowWhenTrue/False/Something` converters cover visibility.

Modify `Plugin.props`: a LongDescription bullet and a `Stacking` tag.

Verify: XAML compiles on macOS; on Windows the options page renders on dark and light themes with the setting off (section present, only Enable active) and on.

Phase 6: docs, checklist, release notes

- `nina/CHANGELOG.md`: `## 1.0.0.32` prose bullets (the join code on the form, live stacking off by default, where frames go, what is never uploaded); mirror to `server/nina_plugin/CHANGELOG.md`; commit titled `N.I.N.A. plugin 1.0.0.32: …` (the author commits).
- `nina/README.md`: step 6 *Live stacking (optional)* under *For people using it*; the `.sln`, `dotnet test`, the second assembly and the native SQLite file under *For people building it*; rows for `Services/Stacking/` and `src/RapidStackerCollab.Core/` in *How it is put together*; licence paragraph.
- Root `starfront/README.md`: a design-decision paragraph in the N.I.N.A. plugin section: the join code lives on the Starfront project and is handed only to telescopes on it; the uploader is a second `ImageSaved` subscriber so the ledger path is untouched; only Starfront lights are uploaded; calibration goes to the last light's project; why `AfterSequenceEnd` was dropped; why the SQLite library is loaded by full path.
- `docs/Starfront Collaboration for NINA.html`: a section on live stacking; reprint the PDF (command in `docs/README.txt`, Windows); copy to `server/nina_plugin/`.
- New `nina/MANUAL_CHECKLIST.md` in upstream's style (tick boxes with *Expected* lines): install (zip contents, no load failure, `e_sqlite3.dll` loaded from the plugin folder); setting off (no folder, no network, no log lines, ledger unchanged); sign in / deny / expire / cancel / sign out; a linked project in the form, check-in, auto-join log line and `links.json`; a container run with the simulator camera, `queue.db` rows, uploads reaching the fake server or a redeployed stack; an XISF night; include-calibration on and off; pause and upload now; the terms-outdated path; token expiry hint; teardown with an upload in flight resumes after restart; toggling the setting while N.I.N.A. runs. Note at the top that the Collab backend is torn down and `FakeCollabServer` is the only automated target.

Verification summary

What	Where	How
Server link	macOS	`cd starfront && python tools/check_server.py`
Core and its tests	macOS	`cd starfront/nina && dotnet build Starfront.sln -c Release && dotnet test tests/RapidStackerCollab.Core.Tests`
Whole plugin compiles, XAML included	macOS	the same build (`EnableWindowsTargeting`)
Package contents, deploy, load	Windows	`dotnet msbuild src/Starfront.Collab/Starfront.Collab.csproj -t:PackagePlugin -p:Configuration=Release` , then `-t:DeployPlugin`
Behaviour with the setting off and on	Windows + N.I.N.A. 3.2	`nina/MANUAL_CHECKLIST.md`
End-to-end upload	Windows + fake server, or a redeployed Collab stack	checklist; the stack redeploy is the author's call

Risks

- **Native SQLite inside N.I.N.A. is unverified** on both sides. Mitigation: preload by full path, the `tasklist /m` checklist step, the `bundle_winsqlite3` fallback (two csproj lines plus `SQLitePCL.raw.SetProvider`).
- **Nullable and warning rules differ** between Core and Starfront; the nested props keep each side's rules, so port files must be written in Starfront's style and never rely on Core's nullable annotations for correctness.
- **`Build-Release.ps1` breaks the day the second csproj lands**; fix it in Phase 2, not later.
- **180-day token, no refresh**: the options page shows the expiry, the dock hints under 14 days, and `AuthRequired` stops uploading, never the night.
- **Duplicate uploads**: the queue is idempotent per (project, path) and the server answers `duplicate` by SHA-256, so a ledger re-credit or a second `Updated` cannot double-send.
- **Headless terms acceptance**: turning the setting on accepts linked projects' current terms; the toggle's hint says so, and a changed version always needs a click.
- **Edit form clearing the code**: owners' telescopes must receive the code; covered in `check_server.py` .
- **XISF**: Core reads and strips XISF (tested with a builder only); the checklist includes one XISF night.
- **Re-sync drift**: Core is copied, not referenced; `SOURCE.md` records the commit and the ported Shell files so a later upstream change can be carried over.

Critical files

- `starfront/nina/src/Starfront.Collab/Services/Runtime.cs` (add `Stacking` , start/stop order)
- `starfront/nina/src/Starfront.Collab/Settings/StarfrontSettings.cs` (new settings)

- `starfront/nina/src/Starfront.Collab/Services/FrameReporter.cs` (hook point via `Lookup`; unchanged)
- `starfront/nina/src/Starfront.Collab/Server/Models.cs`, `Ui/StartCollabVM.cs`, `Ui/StarfrontDockVM.cs`, `Ui/DockTemplates.xaml`, `Ui/OptionsTemplates.xaml`, `Plugin.cs`
- `starfront/nina/build/Plugin.targets`, `build/Build-Release.ps1`, `build/Stage-Plugin.ps1`, `.github/workflows/ci.yml`
- `starfront/server/app.py`, `starfront/tools/check_server.py`
- `Rust/rapid-stacker-collab/nina-plugin/src/RapidStackerCollab.Core/` (copied verbatim) and `tests/RapidStackerCollab.Core.Tests/`
- `Rust/rapid-stacker-collab/nina-plugin/src/RapidStackerCollab.Shell/Hosting/CollabServices.cs` (the graph to mirror), `Intake/FrameIntake.cs`, `Filters/FilterMappingGrid.cs`, `Projects/JoinFlow.cs`, `Imaging/ImagingStateTracker.cs`, `Dock/VerdictPoller.cs` (ported by hand)